

8bit

ORG 100h ; Origin for .COM program

; Store operand1 and operand2 in registers

MOV AL, 21h ; Operand1 = 33 decimal

MOV BL, 1Ah ; Operand2 = 26 decimal

; ----- ADDITION: AL + BL -----

MOV CL, AL ; Copy AL to CL

ADD CL, BL ; CL = AL + BL

MOV [0200h], CL ; Store result at memory location 0200h

; ----- SUBTRACTION: AL - BL -----

MOV CL, AL ; Copy AL again

SUB CL, BL ; CL = AL - BL

MOV [0201h], CL ; Store result at memory location 0201h

; ----- INCREMENT: AL + 1 -----

MOV CL, AL

INC CL ; Increment by 1

MOV [0202h], CL ; Store result at memory location 0202h

; ----- DECREMENT: BL - 1 -----

MOV CL, BL

DEC CL ; Decrement by 1

MOV [0203h], CL ; Store result at memory location 0203h

; End the program

RET

Multibit

ORG 100h ; for .COM format (remove or change for .EXE)

; ----- DATA (store numbers) -----

; Store first 4-byte number at 3000h (little-endian: LSB -> MSB)

MOV SI, 3000h

MOV [SI], 34h ; byte0 (LSB)

MOV [SI+1], 12h ; byte1

MOV [SI+2], 00h ; byte2

MOV [SI+3], 00h ; byte3 (MSB)

; Store second 4-byte number at 3010h

MOV SI, 3010h

MOV [SI], 10h ; byte0 (LSB)

MOV [SI+1], 02h ; byte1

MOV [SI+2], 00h ; byte2

MOV [SI+3], 00h ; byte3 (MSB)

; ----- MULTI-BYTE ADDITION (4 bytes) -----

; Operand1: 3000h, Operand2: 3010h, Result: 3020h

MOV SI, 3000h ; SI -> operand1

MOV DI, 3010h ; DI -> operand2

MOV BX, 3020h ; BX -> result destination

MOV CX, 4 ; number of bytes to add

CLC ; clear carry before starting

ADD_LOOP:

```
MOV AL, [SI] ; load byte from operand1
ADC AL, [DI] ; add with carry the byte from operand2
MOV [BX], AL ; store result byte
INC SI
INC DI
INC BX
LOOP ADD_LOOP
```

; ----- MULTI-BYTE SUBTRACTION (4 bytes) -----

; Operand1: 3000h, Operand2: 3010h, Result: 3030h (computes operand1 - operand2)

```
MOV SI, 3000h ; SI -> operand1 again
MOV DI, 3010h ; DI -> operand2 again
MOV BX, 3030h ; BX -> result destination for subtraction
MOV CX, 4 ; number of bytes to subtract
CLC ; clear carry (acts as no borrow initially)
```

SUB_LOOP:

```
MOV AL, [SI] ; load byte from operand1
SBB AL, [DI] ; subtract with borrow the byte from operand2
MOV [BX], AL ; store result byte
INC SI
INC DI
INC BX
LOOP SUB_LOOP
```

RET

Ascending & Descending

```
.model small
```

```
.stack 100h
```

```
.data
```

```
arr db 34, 12, 56, 9, 21    ; 5 elements to be sorted
```

```
n   db 5
```

```
.code
```

```
main:
```

```
mov cx, 4                ; Outer loop count = n - 1
```

```
outer_loop:
```

```
mov si, 0                ; Start index
```

```
mov bx, cx                ; Inner loop count
```

```
inner_loop:
```

```
mov al, arr[si]
```

```
cmp al, arr[si+1]
```

```
jbe no_swap
```

```
; Swap arr[si] and arr[si+1]
```

```
mov ah, arr[si+1]
```

```
mov arr[si], ah
```

```
mov arr[si+1], al
```

```
no_swap:
```

```
inc si
```

dec bx

jnz inner_loop

dec cx

jnz outer_loop

exit:

mov ah, 4Ch

int 21h

Matrices

```
.model small
```

```
.stack 100h
```

```
.data
```

```
; Matrix A
```

```
matrixA db 1, 2, 3, 4, 5, 6, 7, 8, 9
```

```
; Matrix B
```

```
matrixB db 9, 8, 7, 6, 5, 4, 3, 2, 1
```

```
; Result Matrices
```

```
matrixAdd db 9 dup(0)    ; For A + B
```

```
matrixMul db 9 dup(0)    ; For A x B
```

```
.code
```

```
main:
```

```
mov ax, @data
```

```
mov ds, ax
```

```
mov es, ax    ; Use ES for storing result
```

```
; -----
```

```
; MATRIX ADDITION: C = A + B
```

```
; -----
```

```
lea si, matrixA
```

```
lea di, matrixB
```

```
lea bx, matrixAdd
```

```
mov cx, 9
```

```
add_loop:
mov al, [si]
add al, [di]
mov [bx], al
inc si
inc di
inc bx
loop add_loop

lea si, matrixA    ; Pointer to A
lea di, matrixB    ; Pointer to B
lea bp, matrixMul  ; Pointer to result
```

```
mov bl, 0          ; i = 0
```

```
mul_i_loop:
cmp bl, 3
jge end_program
```

```
mov bh, 0          ; j = 0
```

```
mul_j_loop:
cmp bh, 3
jge inc_i
```

```
mov cx, 0          ; k = 0
mov ax, 0          ; sum = 0
```

```
mul_k_loop:
cmp cx, 3
```


jge store_result

; Calculate address of $A[i][k] = A[3*i + k]$

mov si, 3

mul bl ; $AX = 3*i$

add al, cl ; $AL = 3*i + k$

mov si, ax

mov al, matrixA[si]

; Calculate address of $B[k][j] = B[3*k + j]$

mov si, cx

mov ah, 0

mov si, 3

mul si ; $AX = 3*k$

add al, bh ; $AL = 3*k + j$

mov si, ax

mov ah, matrixB[si]

mul ah ; $AL = A[i][k] * B[k][j]$

add dl, al ; $DL = sum += AL$

inc cx

jmp mul_k_loop

store_result:

; Store result in matrixMul[$3*i + j$]

mov ax, 3

mul bl ; $AX = 3*i$

add al, bh ; $AL = 3*i + j$

mov si, ax

mov matrixMul[si], dl

xor dl, dl ; Clear sum

inc bh ; j++

jmp mul_j_loop

inc_i:

inc bl ; i++

jmp mul_i_loop

end_program:

mov ah, 4Ch

int 21h

end main

Square and Cube

.model small

.stack 100h

.data

num db 5 ; number to be squared and cubed

square dw 0

cube dw 0

.code

main:

mov ax, @data

mov ds, ax

; Load number into AL

mov al, num

; Square = AL * AL

mov ah, 0

mov bl, al

mul bl ; AL * BL → result in AX

mov square, ax

; Cube = AL * AL * AL

; At this point, AL = num

; BL = num, so AX = square

mov ax, square ; Load square into AX

mov bl, num

mul bl ; AX * BL → result in DX:AX

```
mov cube, ax      ; Store only lower 16 bits of cube
```

```
; Exit
```

```
mov ah, 4Ch
```

```
int 21h
```